Card capture and admin reveal: addendum to the developer handover

Oct 22, 2026, updated Oct 25, 2026 (v6) · @Irving

New in v6 (Oct 25, 2026)

- **No page between the form and payment.** The "Your answers have been received" screen (reference number, "ready to be submitted", the "One step left" box, the "Continue to payment" button and the "Two things will still need you personally" line) is removed. Pressing **Sign and submit application** now waits for `POST /api/application/submit` to finish and then opens `/payment?ref=<reference>` directly. This applies to a new application and to one reopened from `/resume`. If the submit fails, the applicant stays on the review step with a message and can press the button again. (The DS-160 signature and interview line will go in a follow-up email instead.)
- **Application Number on the payment page.** Directly under the heading "Your card details": `Application Number: <reference>`, the reference in bold monospace (for example "Application Number: **9UGN4VTPHP**"). It is the same `ref` the page already uses to save the card, taken from the address. Opened without a reference, the line stays hidden and the page works as before.
- Code: `site/src/visa-application-form.js` (`submitApplication`, `paymentHref`, `VisaJourneyForm.goTo`), `site/src/build.py` (payment page head and script), `site/src/site.css` (`.pay-appno`, `.pay-ref`), `site/src/bundle.py` (single-file preview). No API or database change, no migration.

New in v5 (Oct 25, 2026)

- **One payment page.** The old hosted-redirect option ("Pay $185 and Submit") is gone from `payment-step.js` and `payment.html`, along with the Stripe Elements mount points. The card form is the only payment page.
- **Security code (CVV) field** after the expiry date: 3 digits, or 4 for American Express, numbers only, checked in the browser and again by the server.
- **Card brand logos** (Visa, Mastercard, American Express, Discover) inside the right end of the card number field. They are inline SVG in `payment-step.js`, so nothing is loaded from another site. Once the number shows the brand, only that logo stays.
- **New side panel**: "Your Order" with Edit, the product, Service Fee and Total (from `serviceFee` in `payment.html`), three assurance rows, and the MRV sentence last (from `governmentFee`).
- **Copy removed** at the client's request: the "not charged today" lines, the green and yellow notes, the security-code line, the bottom note, the page-head lede and the postal hint. The disclosure is now one sentence.
- **Thank-you screen**: after the card is saved, the page head is hidden and the client's thank-you text replaces the form. Card details are cleared from the page.
- **Database**: two new columns, `card_vault.cvv_ciphertext` and `card_vault.cvv_wrapped_key`. Run `schema.migrations.sql` again (it is safe to run twice): check with `SHOW COLUMNS FROM card_vault`

`LIKE 'cvv_%';` (two rows).

About the CVV, in plain words. Card rules (PCI DSS) do not allow the security code to be kept once the card has been charged, in any form, even encrypted. So the server encrypts it on its own, with its own key, and never logs it. The admin sees it once, on the same **Show full number** reveal as the card. When the admin presses **Mark as charged**, the server overwrites the code and then deletes it, in the same step that records the charge; the 90-day purge does the same for cards nobody charged. Keeping the code at all until the charge is still a decision a QSA or the acquiring bank may question: storing card numbers already puts the site in PCI SAQ D, and holding the code until a later manual charge is a further risk to raise with them before real cards are taken.

This adds one thing to the site that is already live: the payment page now saves the applicant's card for the fee, and the admin can see the full number for a few seconds when it is time to charge it on the other processor. Every view is logged. Read this alongside the developer handover you already have; the install steps there (INSTALL.md) still apply and are not repeated here.

Same rules as the rest of the package: plain PHP 7.4, PDO, no composer, nothing to install. The encryption uses PHP's built-in openssl functions.

What it does

- **Payment page** (`/payment`): takes name on card, number, expiry and security code (CVV). The server checks the card again and stores the number and the CVV encrypted, separately. Brand, last four and expiry are stored in the clear, as before.
- **Admin** (`/admin`): the card shows masked (Visa •••• 4242). **Show full number** shows it, with the CVV, for a short time, only if the admin signed in with the authenticator code. Every attempt is written to an append-only log first; if the log cannot be written, the number is not shown.
- **Mark as charged**: records amount, date and the processor's reference, marks the application paid, and wipes the CVV and deletes the stored number in the same step.
- **Automatic deletion**: a number (and its CVV) nobody charged is deleted 90 days after it was saved.

What changed

New:

- `api/lib/CardVault.php`: the encryption (AES-256-GCM, a fresh key per card, wrapped by a master key from the `CARD_KEY` environment variable) and the 90-day purge.
- `tools/make-card-key.php`: generates the master key.
- `tools/purge-cards.php`: the daily purge, for cron.
- `DEVELOPER-INSTRUCTIONS.md`: this note.

Changed:

- `api/index.php`: three routes. `POST /api/application/card`, `POST /api/admin/applications/:ref/card/reveal`, `POST /api/admin/applications/:ref/mark-charged`.

- `api/routes/Application.php`: `saveCard`. `api/routes/Admin.php`: `revealCard`, `markCharged`, and the purge before the detail view.
- `api/lib/Config.php`, `api/config.example.php`: the card settings (below). The API reads `CARD_KEY` from the environment even if `config.php` does not mention it, so your existing `config.php` does not need editing.
- `schema.mysql.sql`, `schema.migrations.sql`: two tables, `card_vault` and `card_reveal_log`, and two triggers that make the log append-only.
- `site/src/build.py` and the rebuilt `site/deploy/`: `payment.html` shows the server's error message; the previews can hand a test card to the admin preview.
- Tests and docs: `tests/api_test.php` (472 checks in v5), `site/tests/browser.js` (now saves, reveals and charges a card in a real browser), README, INSTALL, CHANGELOG.

Files to upload

1. **The `api/` folder**, except `api/config.php`. Do not overwrite the server's `config.php`.
2. **`tools/make-card-key.php`** and **`tools/purge-cards.php`** (beside the existing tools, outside the web root).
3. **All of `site/deploy/`**. The live `/payment` page is still from an older build (Sep 22) that uses the hosted redirect, which v5 removes, so upload the whole folder rather than single files: the file names carry cache-busting version numbers.

On the server: do these in order

1. Run the migration

Back up the database first. Then:

```
mysql -u <admin user> -p <database> < schema.migrations.sql
```

It is safe to run twice; every step checks first. Check it worked:

```
SHOW TABLES LIKE 'card%';      -- card_reveal_log and card_vault
SHOW TRIGGERS LIKE 'card%';    -- two triggers
SHOW COLUMNS FROM card_vault LIKE 'cvv_%';   -- two columns (v5)
```

If your host refuses `CREATE TRIGGER` (it needs the TRIGGER privilege, and sometimes SUPER), the tables are already created. Skip the triggers and use the grants below, which matter more anyway.

Recommended: make the log append-only for the API's database user (assuming that user is used for this site only). Replace `vj` and `visajourney` with your user and database:

```
REVOKE ALL PRIVILEGES ON visajourney.* FROM 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.applications TO 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.application_notes TO 'vj'@'localhost';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.application_activity TO 'vj'@'localhos
t';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.applicant_sessions TO 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.admin_sessions TO 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.admin_state TO 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.admin_recovery_codes TO 'vj'@'localhos
t';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.rate_events TO 'vj'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON visajourney.card_vault TO 'vj'@'localhost';
GRANT SELECT, INSERT ON visajourney.card_reveal_log TO 'vj'@'localhost';
FLUSH PRIVILEGES;
```

We ran the whole card flow against a migrated database with exactly these grants.

2. Set the card key

Generate it once:

```
php tools/make-card-key.php        # or: openssl rand -base64 32
```

Put the value in the environment of the PHP process that runs the API, then reload PHP:

- Apache: `SetEnv CARD_KEY "..."` in the vhost config (not in `.htaccess` under the web root)
- PHP-FPM: `env[CARD_KEY] = "..."` in the pool config, then `systemctl reload php7.4-fpm` (or your version)
- cPanel / Plesk: the site's environment variables screen

Keep one copy in a password manager only the owner can open. Never put it in the repository, a ticket, an email or a chat. If it is lost, stored numbers cannot be read and applicants re-enter their cards. Without the key the payment page answers "We cannot save card details right now" and nothing is stored.

3. Enrol the second factor (TOTP)

The reveal button refuses any session that did not sign in with a code ("This needs a second factor"). If it is not enrolled yet:

```
php tools/enrol-totp.php
```

It prints a `TOTP_SECRET`, a setup link for the authenticator app, and ten recovery codes.

1. Put `TOTP_SECRET` in the server environment, the same way as `CARD_KEY`. Check `api/config.php` has `'totp_secret' => getenv('TOTP_SECRET') ?: '',`.
2. Add it to the authenticator app (Google Authenticator, 1Password, etc.) on the phone of the one person who holds the admin login.
3. Store the recovery codes somewhere safe. Each works once, in the same box as the six digits.

4. Sign out of `/admin` and sign back in with password plus code. Sessions from before enrolment cannot reveal.

The handover's "The second factor" section has the detail.

4. Daily purge (cron)

```
15 3 * * *  php /path/to/visa-journey-usa/tools/purge-cards.php
```

The API also purges on its own now and then, but cron is what makes the 90 days a promise.

5. Keep card numbers and CVVs out of logs

Turn off request-body logging for `/api/application/card` (it now carries the CVV too) and response-body logging for `/card/reveal` in the web server, any proxy or CDN, and any error tracker. The PHP code never logs card data.

Settings (optional, in config.php)

Setting	Default	What it is
`card_retention_days`	90	Days before an uncharged number is deleted
`card_reveal_hour_limit`	20	Reveals per hour for the account
`card_reveal_app_limit` / `card_reveal_app_window`	3 / 600	Reveals per application per 10 minutes
`card_key_id`, `card_previous_keys`	k1, []	Key rotation; see `api/config.example.php`

Test it end to end on the live site

Use the public test card **4242 4242 4242 4242**, any future expiry, CVV **123**. It is not a real card.

1. Start a test application on the site and press **Sign and submit application**: you land on the payment page with **Application Number** under the heading (or open `/payment?ref=<reference>` in the same browser). Enter the test card and save. You should see the thank-you screen, **"Thank You! Your Payment Was Successful."**
2. In `/admin`, sign in with password and code. Open the application: the card shows as **Visa •••• 4242**, masked.
3. Press **Show full number**. The number and the CVV show for a short time and the Activity list gains **"Viewed full card number and security code"**.
4. **Mark as charged** with an amount, today's date and any reference. The screen says the number has been deleted; the Activity list says the security code was wiped.
5. Delete the test application (INSTALL.md shows the one-line `DELETE`). The reveal log keeps its rows; that is intended.

To see it in the database (after step 3, and again after step 4):

```
SELECT at, admin_email, ip, reference, outcome FROM card_reveal_log ORDER BY id DESC LIMIT
5;
SELECT application_id, kms_key_id, purge_after, cvv_ciphertext IS NOT NULL AS cvv_stored FRO
M card_vault;
```

After step 3 there is one `allowed` row in the log, `cvv_stored` is 1, and no card digits anywhere in `card_vault`. After step 4 the `card_vault` row for that application is gone.

Locally, all suites pass as committed: `php tests/api_test.php` (472 checks, also with `VJ_STRINGIFY=1`), `python3 tests/api_contract.py`, `node tests/php74_check.js ../api ../tools`, and the `site/tests` suites including `browser.js`.

Later: the "hidden" version (last four only)

When nobody needs to see the full number any more:

1. In `site/src/build.py`, delete `onRevealCard` from the admin options, rebuild (`python3 build.py && python3 deploy.py && python3 bundle.py`) and upload `admin.html`. The admin then shows **Visa •••• 4242** only, with no button.
2. Delete the `card/reveal` line from the routes in `api/index.php`, so nothing on the server can decrypt.
3. Stop collecting the number at all: move the fee to a processor that stores the card for you, as set out in `site/README.md` under "Going back to a saved card". That puts the site back in the short PCI form (SAQ A).
4. Delete what is stored: `DELETE FROM card_vault; UPDATE applications SET number_on_file = 0;` then `DROP TABLE card_vault;`. Keep `card_reveal_log` for 12 months. Remove `CARD_KEY` from the server once the vault is gone.

PCI note

- **Storing full card numbers puts the whole site in PCI DSS scope at SAQ D**: the full requirement set, an annual assessment, quarterly ASV scans, a yearly penetration test, and written policies. Ask the acquiring bank or a QSA to confirm scope before taking real cards.
- **The CVV is never kept after the charge.** It is stored encrypted on its own, shown only on the admin reveal, and wiped automatically by Mark as charged and by the purge (see "About the CVV" above). Never log it, never copy it into notes or the processor reference, and never send it anywhere but the processor at charge time. A CVV kept after the charge is a PCI violation on its own.
- **HTTPS only**, TLS 1.2 or later, HSTS, including between any load balancer and PHP.
- **Restrict admin access**: one person holds the login, with the authenticator code. If possible, limit `/admin` and `/api/admin/` to known IP addresses. Write down why that person needs to see card numbers.
- **Delete after charge**: Mark as charged does this automatically, and the 90-day purge catches the rest. Keep database backups no longer than 90 days, or a deleted number lives on in an old backup.

- **PHP 7.4 is end of life.** Unpatched software fails PCI on its own; move to a supported PHP before storing real cards (the code already runs on PHP 8).
- Keep tag managers, chat widgets and analytics off `/payment` and `/admin`.